INGENIERÍA DE SOFTWARE CON IA · 2026

Desarrollo Agéntico y Harness Engineering

Contexto, guardarraíles, memoria persistente y metodología profesional para dirigir agentes de IA

Impacto Real de la IA en 2026

90%

Adopción diaria

Desarrolladores programando con IA

45%

Código generado

Nuevo software escrito por agentes

30%

Confianza ciega

Resultados válidos sin supervisión

> El mercado ya no paga por teclear sintaxis, sino por orquestar arquitectura y validar criterio.

01 · CAMBIO DE PARADIGMA

Vibe Coding vs. Ingeniería de Software

⚠ Vibe Coding

Intuición pura: Pedir código rápido sin diseño

Cero revisión: Aceptar cambios a ciegas

Deuda técnica: Sistemas frágiles e inmantenibles

Ingeniería Agéntica

Arquitectura clara: Reglas definidas antes de actuar

Control humano: Supervisión en puntos críticos

Sostenibilidad: Pruebas, trazabilidad y evolución

Acotar el Indeterminismo de la IA

Un LLM no piensa ni razona verdad: predice el siguiente token por probabilidad.

La temperatura introduce aleatoriedad y la ventana de contexto limitada provoca olvidos o alucinaciones si se satura.

Mecanismos de Acotación

Contexto curado: Solo información relevante y vigente

Guardarraíles: Prohibiciones explícitas y permisos

Validación determinista: Tests, linters y compiladores

Sesiones compactas: Evitar degradación por tokens

02 · ARQUITECTURA DE EJECUCIÓN

Del Chat Pasivo al Bucle Autónomo del Agente



A diferencia del chat (un turno con copy-paste), el agente opera dentro de un **Arnés (Harness)** con acceso real al entorno.

Context Engineering y el Rol Clave de Markdown

Curaduría Activa

Seleccionar y actualizar solo lo relevante.

Jerarquía Nativa LLM

Títulos y listas guían la atención.

Código Versionable

Texto plano ligero auditado en Git.

Dónde es Estratégico Usar Markdown

Artefacto	Propósito en el Harness	Beneficio Clave
`Prompts Estructurados`	Separar Rol, Contexto, Tarea y Reglas	Elimina ambigüedad semántica
`AGENTS.md`	Constitución y guardarraíles del proyecto	Lectura automática por defecto
`MEMORY.md`	Estado vivo, decisiones y errores previos	Continuidad entre sesiones
`Specs y Skills`	Criterios de aceptación y flujos reutilizables	Escala conocimiento del equipo

AGENTS.md: Constitución del Proyecto

```
# AGENTS.md · Regla: 30-40 líneas máx.
## Stack y estructura
Tecnologías clave y carpetas críticas.
## Convenciones y trampas
Patrones obligatorios y errores frecuentes.
## Límites (Siempre / Pregunta / Nunca)
Fronteras de autonomía del agente.
## Verificación
Comandos exactos de test y lint.
```

Principios de Diseño

Brevedad estricta: Máximo 40 líneas para no saturar tokens

Generación inicial: El comando /init analiza y crea la base

Trampas de dominio: Documentar lo que no se deduce del código

Semáforo de límites: Definir qué requiere aprobación humana

MEMORY.md: Memoria Viva del Proyecto

- **Estado actual:** Qué versión funciona y qué módulos están activos
- **Decisiones y porqué:** Arquitectura elegida para no revertir criterios
- **Aprendizajes:** Bugs resueltos y errores a evitar en el futuro
- **Poda constante:** Máximo ~50 líneas; promover reglas fijas a AGENTS.md

> **Regla en AGENTS.md:** "Al empezar lee MEMORY.md; al terminar cada tarea actualízalo sin guardar secretos ni claves."

Harness Avanzado: Comandos, Skills y MCP

/comandos

Comandos Custom

Atajos repetibles con prompts predefinidos.

@skills

Agent Skills

Capacidades modulares bajo demanda.

MCP Protocol

Model Context Protocol

Conexión estándar a APIs y datos.

Modo Plan vs. Modo Build

Modo Plan (Solo Lectura)

Propósito: Analizar, detectar riesgos y proponer pasos
Ideal para: Tareas grandes, refactors y arquitectura
Seguridad: No toca archivos hasta tu aprobación

Modo Build (Ejecución)

Propósito: Escribir código, ejecutar comandos y tests

Ideal para: Correcciones obvias y planes ya aprobados
Control: Auditable con /diff y reversible con /undo

Flujo de Trabajo y Puntos de Control Humano

FASE 1 · PREPARAR

Contexto y Reglas

Definir AGENTS.md y spec en Markdown.

FASE 2 · CONTROL

Modo Plan

Validar enfoque y casos límite antes de tocar código.

FASE 3 · EJECUTAR

Iteración Pequeña

Modo Build con cambios acotados y sesión limpia.

FASE 4 · CONTROL

Auditar y Persistir

Revisar /diff, tests y actualizar MEMORY.md.

Control obligatorio siempre: Arquitectura, permisos/seguridad, esquemas de datos, revisión de /diff y cierre de memoria.

SÍNTESIS · EL NUEVO PROGRAMADOR

La IA ejecuta el código. Tú gobiernas el sistema.

Fundamentos de ingeniería + Context Engineering + Guardarraíles deterministas